

AI Security Operations & Boundary Protection

Practical implementation of AI Security skills — covering guardrails, injection defense, exfiltration prevention, and AI-assisted automation for building resilient, production-safe AI systems.

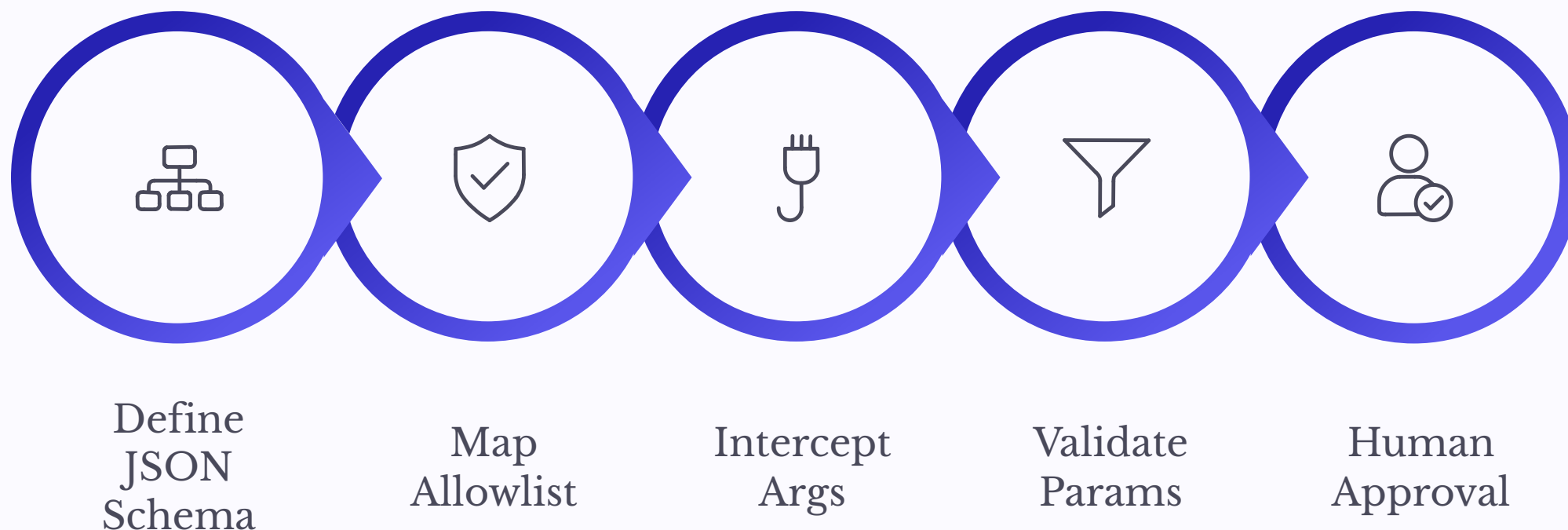
MAISAM SHAIKH IBRAHIM



Made with GAMMA

Securing Agentic AI Tool Invocation

Intercept and secure the execution layer of autonomous AI agents to prevent unauthorized shell or tool execution.



This layered approach ensures every tool call is validated, scoped, and approved before execution — eliminating unauthorized access at the agent boundary.

Defending Against Indirect Prompt Injection

Neutralize adversarial commands embedded in untrusted external text before they reach the model.

→ Isolate Raw Inputs

Wrap external data in structural boundaries and XML tags to prevent context bleed.

→ Real-Time Signature Scanning

Scan for known injection payloads using regex parsers at ingestion.

→ Secondary Validation Layer

Run an isolation verification loop to catch bypass attempts.

→ Truncate & Alert

Limit context window size and generate real-time log alerts for suspect strings.

Automated PII Masking & Data Redaction



Automatically detect and scrub sensitive identifiers from prompts before transmission — preserving data privacy at the ingestion layer.

01

Intercept at Proxy

Capture prompts before they reach the model.

02

Pattern Scan

Detect credentials, credit cards, and identity metrics via regex.

03

Redact & Forward

Swap matches with `[REDACTED_DATA]` and send only the sanitized prompt.

04

Block Training Storage

Ensure private data never enters centralized training vectors.

SKILL 5

Post-Inference System Prompt Protection

Block model responses that attempt to leak secret system rules or backend configurations to end-users.

Capture Output

Intercept raw model output before it reaches the user interface.

Keyword Evaluation

Scan against application-level keywords and baseline instruction patterns.

Detect Leakage

Identify phrase patterns matching hidden system prompt layouts.

Block & Replace

Drop the packet instantly and serve a safe, compliant fallback statement.

Outbound Response Validation & Safety Filters

Analyze generated responses in real-time to catch and suppress model hallucinations or non-compliant content before delivery.

1

Route Output

Pass model output through a localized safety alignment layer.

2

Scan Content

Detect toxic phrases, syntax breaks, and semantic violations.

3

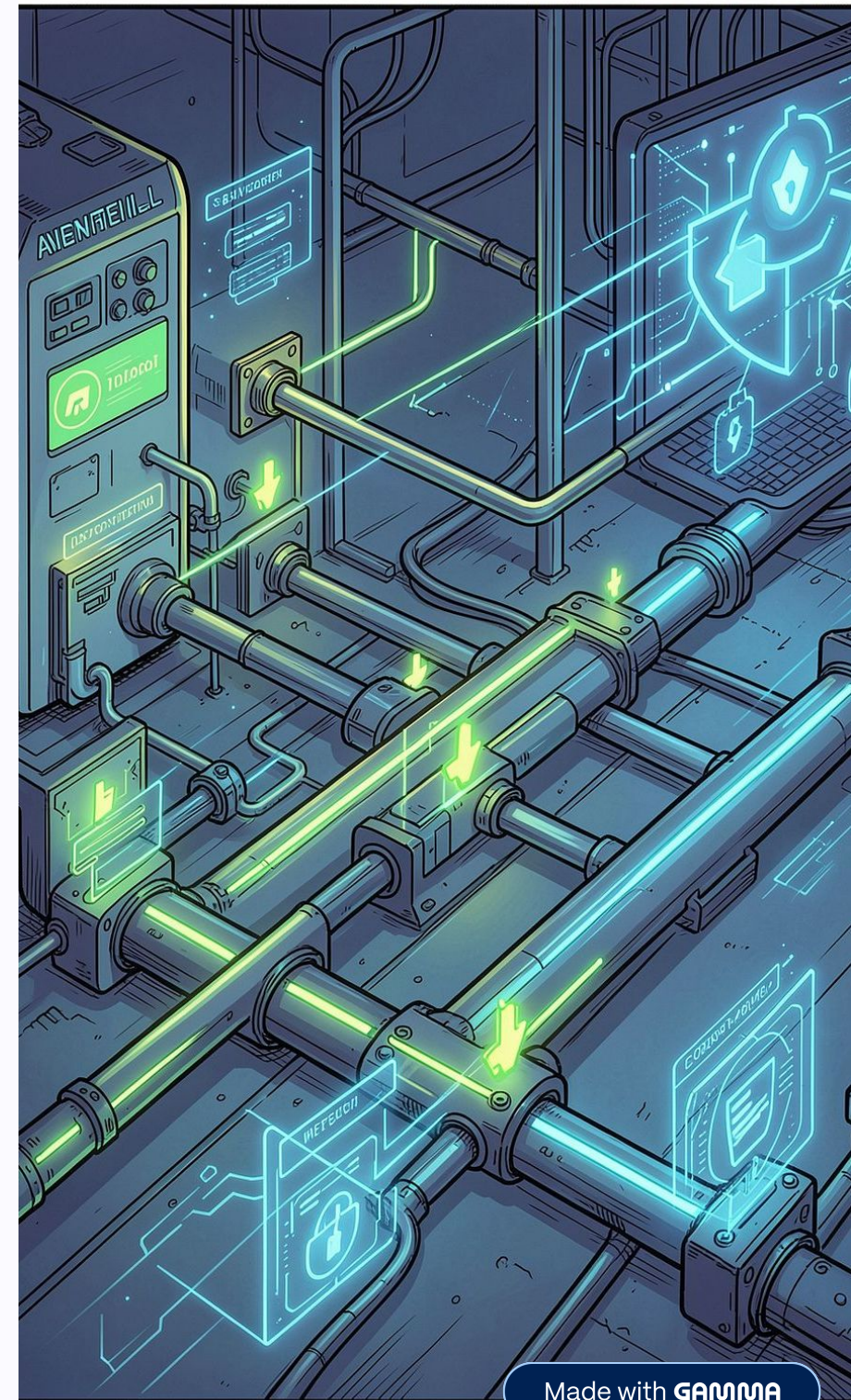
Check Metrics

Validate semantic output against safe structural boundaries.

4

Mitigate & Serve

Trigger fallback code if safety thresholds drop; serve a standardized error message.



User Rate Limiting & Token Throttling

Protect server availability from API flooding, resource exhaustion, and automated botnet script attacks at the perimeter.



Trace Request Streams

Identify clients via IP addresses or session tokens.

Enforce Thresholds

Cap at defined limits (e.g., 5 queries/minute) and drop spikes.

Cache & Mask

Cache repetitive inquiries locally and mask server errors to hide system paths.

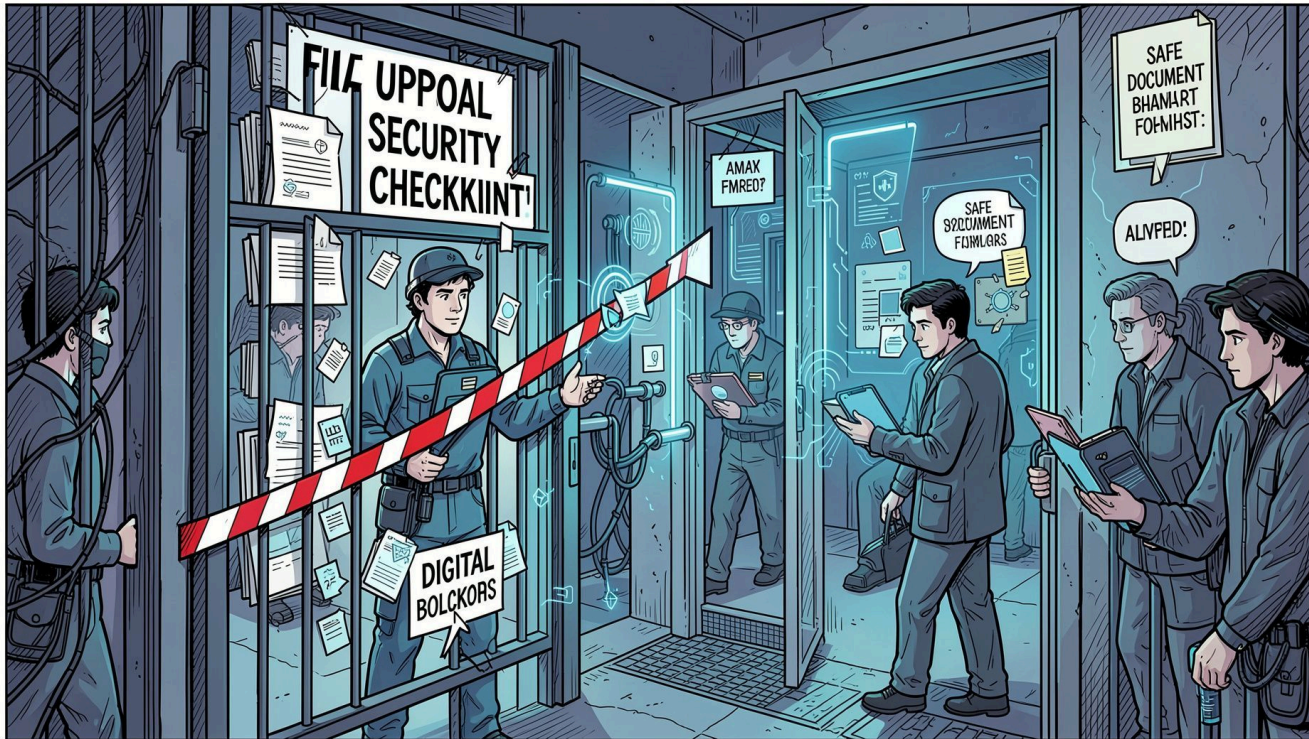
Deny-by-Default Capability Mapping

Enforce least privilege across an agent's technical toolkit — preventing scope creep and unauthorized resource access.



Every tool invocation is checked against a static manifest. Any unmapped path triggers an immediate execution kill and generates an audit log — ensuring the agent never exceeds its authorized scope.

Rigid File Ingestion Extension Enforcement



Secure data-parsing pipelines against binary exploits and remote code execution by enforcing strict file-type controls at the boundary.

1 Catch at Boundary

Intercept all uploads before processing begins.

2 Evaluate Metadata

Extract and validate file extension types and headers.

3 Block Binaries

Reject `.exe` and other executable formats instantly.

4 Sanitize & Load

Allow only `.txt` or `.json`; strip hidden scripting before loading.

Structural Error Masking & Exception Management

Catch runtime errors to prevent deep system details from leaking to users or attackers.



Security Wrapper

Wrap core scripts with a catch-all exception handler.



Suppress Stack Traces

Intercept crashes and stop raw traces from printing.



Private Admin Logs

Route sensitive telemetry to secure admin-only logs.



Generic Error Codes

Serve clean, non-revealing errors to the user interface.

 **Key Principle:** Never expose system architecture, file paths, or database details through error messages — attackers use them for reconnaissance.